# MATH 128 - HW7 - Jie Wen

# Problem 1: Problems from the textbook, chapter 2.5

## 1

MATLAB codes:

```
clear; clc;

% --- Parameters
numHats = 5;           % need hat{p}_0 through hat{p}_4
% To form hat{p}_n we need p_n, p_{n+1}, p_{n+2}  -> generate up to p_{numHats+2}
N = numHats + 2;

% --- Generate the base sequence p_n
p = zeros(N+1,1);      % store p_0 .. p_{N}
p(1) = 0.5;            % p_0
for n = 2:(N+1)
    p(n) = cos(p(n-1));
end

% --- Aitken Δ^2 accelerated sequence hat{p}_n for n = 0..4
hatp = nan(numHats,1);
for n = 1:numHats
    % using 1-based indices: p(n) -> p_{n-1}, p(n+1) -> p_n, p(n+2) -> p_{n+1}
    pn   = p(n);
    pn1  = p(n+1);
    pn2  = p(n+2);
    denom = pn2 - 2*pn1 + pn;
    if abs(denom) < 1e-14
        warning('Δ^2 p_n is (near) zero at n-1 = %d; Aitken term ill-defined.', n-1);
        hatp(n) = NaN;
    else
        hatp(n) = pn - ( (pn1 - pn)^2 ) / denom;
    end
end

% --- Print results
fprintf('Base sequence p_n (n = 0..%d):\n', N);
for n = 0:N
    fprintf('  p_%d = %.15f\n', n, p(n+1));
end
fprintf('\nFirst five Aitken-accelerated terms hat{p}_n:\n');
for n = 0:(numHats-1)
    fprintf('  hat{p}_%-2d = %.15f\n', n, hatp(n+1));
end
```

Output:

```
Base sequence p_n (n = 0..7):
  p_0 = 0.500000000000000
  p_1 = 0.877582561890373
  p_2 = 0.639012494165259
  p_3 = 0.802685100682335
  p_4 = 0.694778026788006
  p_5 = 0.768195831282016
  p_6 = 0.719165445942419
  p_7 = 0.752355759421527

First five Aitken-accelerated terms hat{p}_n:
  hat{p}_0  = 0.731385186382582
  hat{p}_1  = 0.736086691713017
  hat{p}_2  = 0.737652871396400
  hat{p}_3  = 0.738469220876263
  hat{p}_4  = 0.738798065173590
```

---

## 4

MATLAB codes:

```
clear; clc;

% Define g(x)
g = @(x) 1 + (sin(x)).^2;

% Initial value p0^(0)
p0 = 1.0;

% --- First Steffensen step: compute p0^(1) from p0^(0)
[p01, p1a, p2a] = steffensen_step_fp(g, p0);

% --- Second Steffensen step: compute p0^(2) from p0^(1)
[p02, p1b, p2b] = steffensen_step_fp(g, p01);

% Display results
fprintf('p0^(0) = %.15f\n', p0);
fprintf('p0^(1) = %.15f\n', p01);
fprintf('p0^(2) = %.15f\n', p02);

% -------- Helper function (file-local or separate .m file) --------
function [p_new, p1, p2] = steffensen_step_fp(g, p0)
%STEFFENSEN_STEP_FP One fixed-point Steffensen step.
%   Inputs:
%       g   : fixed-point function handle
%       p0  : current iterate (scalar)
%   Outputs:
%       p_new : accelerated next iterate (scalar)  -> this is p0^(i)
%       p1, p2: the two plain fixed-point iterates g(p0), g(g(p0))

    p1 = g(p0);
    p2 = g(p1);
    denom = (p2 - 2*p1 + p0);

    % Guard against (near-)zero denominator (as textbooks warn)
    if abs(denom) < 1e-15
        error('Steffensen denominator nearly zero; cannot proceed safely.');
    end

    p_new = p0 - (p1 - p0)^2 / denom;
end
```

Outputs:

```
p0^(0) = 1.000000000000000
p0^(1) = 2.152904628999380
p0^(2) = 1.873464043514280
```

---

# 6

MATLAB code:

```
clear; clc;

% Inputs
p0_0  = 1.0;             % p0^(0)
p1_0  = sqrt(2);         % p1^(0)
p0_1  = 2.7802;          % p0^(1) provided

% Compute Δ and Δ^2
Delta  = p1_0 - p0_0;
Delta2 = Delta^2;        %#ok<*NASGU> (explicit for clarity)

% Back-solve for p2^(0):
% p0^(1) = p0^(0) - (p1^(0)-p0^(0))^2 / ( p2^(0) - 2*p1^(0) + p0^(0) )
% => p2^(0) = - (Δ^2)/(p0^(1)-p0^(0)) + 2*p1^(0) - p0^(0)
p2_0 = - (Delta^2) / (p0_1 - p0_0) + 2*p1_0 - p0_0;

fprintf('p2^(0) = %.12f\n', p2_0);
```

Output:

```
p2^(0) = 1.732048698022
```

# 9

MATLAB codes:

```
clear; clc;

TOL = 1.0e-4;
MAXIT = 1000;

% Target: sqrt(3)
f  = @(x) x.^2 - 3;
g  = @(x) 0.5*(x + 3./x);   % Heron's (Newton) fixed-point map for sqrt(3)

%% 1) Steffensen's Method (with base iteration g) starting at p0 = 2
p0 = 2;
[p_steff, it_steff] = steffensen(g, p0, TOL, MAXIT);

%% 2) Fixed-Point Iteration (using the same g) starting at p0 = 2
[p_fpi, it_fpi] = fixed_point(g, p0, TOL, MAXIT);

%% 3) Bisection on f(x)=x^2-3 over [1,2]
a = 1; b = 2;
[p_bis, it_bis] = bisection(f, a, b, TOL, MAXIT);

%% Report
true_val = sqrt(3);

fprintf('================ Results (tolerance = %.1e) ================\n', TOL);
fprintf('True sqrt(3)  : %.15f\n\n', true_val);

fprintf('Steffensen    : p = %.15f,  |p - sqrt(3)| = %.3e,  iterations = %d\n', ...
    p_steff, abs(p_steff-true_val), it_steff);

fprintf('Fixed-Point   : p = %.15f,  |p - sqrt(3)| = %.3e,  iterations = %d\n', ...
    p_fpi, abs(p_fpi-true_val), it_fpi);

fprintf('Bisection     : p = %.15f,  |p - sqrt(3)| = %.3e,  iterations = %d\n', ...
    p_bis, abs(p_bis-true_val), it_bis);
fprintf('============================================================\n');

%% ----------------- Local function definitions -----------------

function [p, it] = steffensen(g, p0, tol, maxit)
% Steffensen's Method: accelerates fixed-point iteration x_{k+1}=g(x_k)
% using Aitken's Δ^2:
%   p_new = p0 - (p1 - p0)^2 / (p2 - 2*p1 + p0),
% where p1 = g(p0), p2 = g(p1). (Two g-evals per iteration.)
    it = 0; p = p0;
    while it < maxit
        p1 = g(p);
        p2 = g(p1);
        denom = (p2 - 2*p1 + p);
        if denom == 0
            % As suggested in many texts: fall back to p2 if Δ^2 == 0
            p_new = p2;
        else
            p_new = p - (p1 - p)^2 / denom;
        end
        it = it + 1;
        if abs(p_new - p) < tol
            p = p_new; return;
        end
        p = p_new;
    end
end

function [p, it] = fixed_point(g, p0, tol, maxit)
% Basic fixed-point iteration p_{k+1} = g(p_k) with successive-difference stop.
    it = 0; p = p0;
    while it < maxit
        p_new = g(p);
        it = it + 1;
        if abs(p_new - p) < tol
            p = p_new; return;
        end
        p = p_new;
    end
end

function [c, it] = bisection(f, a, b, tol, maxit)
% Classic bisection with interval-width stopping: stop when (b-a)/2 < tol.
```

```
    fa = f(a); fb = f(b);
    if fa*fb > 0
        error('f(a) and f(b) must have opposite signs.');
    end
    it = 0;
    while it < maxit
        c = (a + b)/2;
        fc = f(c);
        it = it + 1;
        if (b - a)/2 < tol
            return
        end
        if fa*fc <= 0
            b = c; fb = fc;
        else
            a = c; fa = fc;
        end
    end
end
```

```
================ Results (tolerance = 1.0e-04) ================
True sqrt(3)  : 1.732050807568877

Steffensen    : p = 1.732050807568877,  |p - sqrt(3)| = 0.000e+00,  iterations = 3
Fixed-Point   : p = 1.732050810014727,  |p - sqrt(3)| = 2.446e-09,  iterations = 3
Bisection     : p = 1.731994628906250,  |p - sqrt(3)| = 5.618e-05,  iterations = 14
===============================================================
```

---

## 15

Let $e_n := p_n - p$.
"Superlinear convergence" means $\lim_{n\to\infty} \frac{|e_{n+1}|}{|e_n|} = 0$ (assuming $e_n \neq 0$ for all large $n$; if the sequence hits $p$ exactly at some finite index, the statement is vacuous thereafter).

Then

$$\frac{|p_{n+1} - p_n|}{|p_n - p|} = \frac{|e_{n+1} - e_n|}{|e_n|} = \left|\frac{e_{n+1}}{e_n} - 1\right|.$$

Since $\frac{|e_{n+1}|}{|e_n|} \to 0$, it follows that $\frac{e_{n+1}}{e_n} \to 0$, hence

$$\lim_{n\to\infty} \frac{|p_{n+1} - p_n|}{|p_n - p|} = |0 - 1| = 1.$$

---

## 17

MATLAB codes:

```
function q1_17
% Aitken on Taylor partial sums of e^x at x=1
% Produces: (i) quick check for part (a), (ii) table of hat{p}_0...hat{p}_8, (iii) brief verdict for (c).

    x  = 1.0;
    N  = 12;                % compute a few extra terms to be safe
    p  = taylor_partial_exp(x, N);     % p(n+1) = P_n(x)
    ee = exp(x);

    % ---- (a) numeric check of the hypothesis limit < 1 (it should tend to 0)
    rat = abs((p(3:end)-ee))./abs((p(2:end-1)-ee));   % ratio |E_{n+1}|/|E_n| for n>=1
    fprintf('Part (a): last few ratios |E_{n+1}|/|E_n| ≈ ');
    fprintf('%.3e ', rat(end-4:end)); fprintf('\n');

    % ---- (b) Aitken Δ^2: \hat p_n = p_n - (Δp_n)^2 / Δ^2 p_n
    % We can compute for n = 0..8 (need p_0..p_10)
    K = 9; % we need hat{p}_0 ... hat{p}_8
    hat = nan(K,1);
    for n = 1:K
        pn   = p(n);
        dp   = p(n+1) - p(n);
        d2p  = p(n+2) - 2*p(n+1) + p(n);
        if abs(d2p) < 1e-16
            hat(n) = NaN;   % guard; extremely unlikely here
        else
```

```
            hat(n) = pn - (dp*dp)/d2p;
        end
    end

    % ---- Print table
    fprintf('\nPart (b): Aitken Δ^2 on p_n = sum_{k=0}^n x^k/k! at x=1\n');
    fprintf('  n        p_n                 |e - p_n|         hat_p_n             |e - hat_p_n|\n');
    fprintf('-------------------------------------------------------------------------------------\n');
    for n = 0:K-1
        pn   = p(n+1);
        hp   = hat(n+1);
        fprintf('%3d  % .15f   %.3e     % .15f   %.3e\n', ...
            n, pn, abs(ee - pn), hp, abs(ee - hp));
    end

    % ---- (c) simple acceleration verdict using error ratios
    err_p   = abs(p(1:K)   - ee);
    err_hat = abs(hat(1:K) - ee);
    speedup = err_p ./ max(err_hat, eps);
    fprintf('\nPart (c): median(|e - p_n| / |e - hat_p_n|) over n=0..8 ≈ %.1f\n', median(speedup));
    fprintf('Verdict: Aitken accelerates convergence strongly here (see errors).\n');

end

% ===== helper: stable Taylor partial sums for e^x =====
function p = taylor_partial_exp(x, N)
% Returns vector p with p(n+1) = sum_{k=0}^n x^k/k!
    p = zeros(N+1,1);
    term = 1.0;     % k=0 term
    s = term;
    p(1) = s;       % P_0(x)
    for n = 1:N
        term = term * (x / n);  % x^n/n!
        s = s + term;
        p(n+1) = s;             % P_n(x)
    end
end
```

Outputs:

```
Part (a): last few ratios |E_{n+1}|/|E_n| ≈ 1.098e-01 9.903e-02 9.017e-02 8.277e-02 7.648e-02

Part (b): Aitken Δ^2 on p_n = sum_{k=0}^n x^k/k! at x=1
  n        p_n                 |e - p_n|         hat_p_n             |e - hat_p_n|
-------------------------------------------------------------------------------------
  0   1.000000000000000   1.718e+00      3.000000000000000   2.817e-01
  1   2.000000000000000   7.183e-01      2.750000000000000   3.172e-02
  2   2.500000000000000   2.183e-01      2.722222222222222   3.940e-03
  3   2.666666666666667   5.162e-02      2.718750000000000   4.682e-04
  4   2.708333333333333   9.948e-03      2.718333333333333   5.150e-05
  5   2.716666666666666   1.615e-03      2.718287037037037   5.209e-06
  6   2.718055555555555   2.263e-04      2.718282312925170   4.845e-07
  7   2.718253968253968   2.786e-05      2.718281870039683   4.158e-08
  8   2.718278769841270   3.059e-06      2.718281831765628   3.307e-09

Part (c): median(|e - p_n| / |e - hat_p_n|) over n=0..8 ≈ 193.2
Verdict: Aitken accelerates convergence strongly here (see errors).
```

## (a) Verifying Theorem 2.14's hypothesis

For a fixed $x$, the Taylor remainder of $e^x$ about 0 in Lagrange form is

$$e^x - P_n(x) = R_n(x) = e^{\xi}\frac{x^{n+1}}{(n+1)!} \quad (\xi \text{ between } 0 \text{ and } x).$$

Hence

$$\frac{|p_{n+1} - e^x|}{|p_n - e^x|} = \frac{e^{\xi_{n+1}}|x|^{n+2}/(n+2)!}{e^{\xi_n}|x|^{n+1}/(n+1)!} = \frac{e^{\xi_{n+1}-\xi_n}|x|}{n+2} \longrightarrow 0 < 1.$$

So $p_n = P_n(x)$ satisfies the hypothesis "$\lim_{n\to\infty} \frac{p_{n+1}-p}{p_n-p} < 1$" (indeed, the limit is 0), making Aitken's $\Delta^2$ applicable.

## (b) Aitken $\Delta^2$ values at $x = 1$

Using the standard Aitken transform

$$\hat{p}_n = p_n - \frac{(p_{n+1} - p_n)^2}{p_{n+2} - 2p_{n+1} + p_n},$$

(which is equivalent to $\hat{p}_n = \frac{p_n p_{n+2} - p_{n+1}^2}{p_n - 2p_{n+1} + p_{n+2}}$), the program produced:

$\hat{p}_0 = 3.000000000000000,$
$\hat{p}_1 = 2.750000000000000,$
$\hat{p}_2 = 2.722222222222222,$
$\hat{p}_3 = 2.718750000000000,$
$\hat{p}_4 = 2.718333333333333,$
$\hat{p}_5 = 2.718287037037037,$
$\hat{p}_6 = 2.718282312925170,$
$\hat{p}_7 = 2.718281870039683,$
$\hat{p}_8 = 2.718281831765628.$

Each value was computed from three consecutive Taylor partial sums ($p_n = \sum_{k=0}^{n} 1/k!$).

### (c) Does it accelerate?

Yes — decisively. Based on the output table:

- Typical errors drop by ~2–3 orders of magnitude after Aitken.
  Example at $n = 5$: $|e - p_5| \approx 1.615 \times 10^{-3}$ vs $|e - \hat{p}_5| \approx 5.209 \times 10^{-6}$.
- The median speedup, median$\left(\frac{|e-p_n|}{|e-\hat{p}_n|}\right) \approx 193.2$ for $n = 0, \ldots, 8$, quantifies the gain.
- This matches the theory that Aitken accelerates sequences that are (asymptotically) linear; although $P_n(1)$ already converges superlinearly (ratio $\to 0$), the tail still behaves "geometric-like" enough for Aitken to strip the dominant error term and yield dramatic improvement.

---

## Problem 2: Newton's method

### (a)

For $f(x) = \frac{x}{\sqrt{1+x^2}} = x(1+x^2)^{-1/2}$, the derivative is

$$f'(x) = (1+x^2)^{-1/2} + x \cdot (-\tfrac{1}{2})(1+x^2)^{-3/2} \cdot (2x) = (1+x^2)^{-1/2} - x^2(1+x^2)^{-3/2} = \frac{(1+x^2) - x^2}{(1+x^2)^{3/2}} = \frac{1}{(1+x^2)^{3/2}}.$$

Newton's method updates by $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$.
Plugging the expressions in:

$$x_{k+1} = x_k - \frac{\frac{x_k}{\sqrt{1+x_k^2}}}{\frac{1}{(1+x_k^2)^{3/2}}} = x_k - x_k \cdot \frac{(1+x_k^2)^{3/2}}{\sqrt{1+x_k^2}} = x_k - x_k(1+x_k^2) = x_k - x_k - x_k^3 = -x_k^3.$$

So the Newton map is exactly $x_{k+1} = -x_k^3$.

Now analyze convergence. From the recurrence, $|x_{k+1}| = |x_k|^3$.
Hence

$$|x_k| = |x_0|^{3^k}.$$

- If $|x_0| < 1$: then $0 < |x_0|^{3^k} \to 0$ super-fast. Thus $x_k \to 0$. In fact, the error satisfies $e_{k+1} = |x_{k+1}| = |x_k|^3 = e_k^3$, i.e., **cubic** order for this specific problem (even faster than the usual quadratic local rate for Newton at simple roots).
- If $|x_0| = 1$: then $x_1 = -1, x_2 = 1, x_3 = -1, \ldots$; it **oscillates** and does not converge.
- If $|x_0| > 1$: then $|x_k| = |x_0|^{3^k} \to \infty$; the iterates **diverge**.

Therefore, Newton's method **succeeds iff** $|x_0| < 1$ for this $f$.

### (b)

MATLAB codes:

```
function q2
    close all; clc;

    % Problem setup
    f  = @(x) x./sqrt(1+x.^2);
    fp = @(x) 1./(1+x.^2).^(3/2);  % not strictly needed since we have closed-form map
    newton_map = @(x) x - f(x)./fp(x);     % equals -x.^3
    g = @(x) -x.^3;                        % identical map

    % Parameters
    Niter = 8;           % how many iterates to show
    x0_list = [0.5, 1.5];
```

```
    for case_id = 1:numel(x0_list)
        x0 = x0_list(case_id);
        [xs, fs] = iterate_newton(newton_map, f, x0, Niter);

        fprintf('\n==== Case %d: x0 = %.6g ====\n', case_id, x0);
        disp(table((0:Niter)', xs(:), abs(xs(:)), fs(:), ...
            'VariableNames', {'k','xk','abs_xk','f_xk'}));

        % --- Cobweb plot for g(x) = -x^3
        figure('Name', sprintf('Cobweb: x0=%.3g', x0), 'Color','w');
        subplot(1,2,1);
        cobweb_plot(g, x0, Niter, [-2,2]);  % domain for plotting
        title(sprintf('Cobweb for g(x)=-x^3, x_0=%.3g', x0), 'Interpreter','none');

        % --- |x_k| vs k (semilogy)
        subplot(1,2,2);
        semilogy(0:Niter, abs(xs), 'o-','LineWidth',1.5,'MarkerSize',6); grid on;
        xlabel('k'); ylabel('|x_k|');
        title('|x_k| vs k (semilogy)');
    end

    % Side-by-side sequence plots to compare cases
    figure('Name','Sequence comparison','Color','w');
    hold on; grid on;
    colors = lines(numel(x0_list));
    for case_id = 1:numel(x0_list)
        x0 = x0_list(case_id);
        xs = iterate_newton(newton_map, f, x0, Niter);
        semilogy(0:Niter, abs(xs), 'o-','LineWidth',1.5,'MarkerSize',6, ...
            'Color', colors(case_id,:));
    end
    legend(arrayfun(@(v) sprintf('x_0=%.3g', v), x0_list, 'UniformOutput', false), ...
           'Location','southwest');
    xlabel('k'); ylabel('|x_k|'); title('|x_k| vs k for both initial guesses');
end

function [xs, fs] = iterate_newton(newton_map, f, x0, Niter)
    xs = zeros(Niter+1,1);
    fs = zeros(Niter+1,1);
    xs(1) = x0;
    fs(1) = f(x0);
    for k = 1:Niter
        xs(k+1) = newton_map(xs(k));  % equals -xs(k)^3
        fs(k+1) = f(xs(k+1));
    end
end

function cobweb_plot(g, x0, N, xrange)
    % Draw y = g(x), y = x, and the cobweb from x0 for N steps.
    if nargin < 4, xrange = [-2, 2]; end
    x = linspace(xrange(1), xrange(2), 2000);
    plot(x, g(x), 'LineWidth',1.6); hold on; grid on;
    plot(x, x, 'k--', 'LineWidth',1.2);
    xlabel('x'); ylabel('y');
    axis equal; axis([xrange(1) xrange(2) xrange(1) xrange(2)]);

    % Cobweb lines:
    xk = x0;
    for k = 1:N
        yk = g(xk);
        % vertical: (xk, xk) -> (xk, g(xk))
        plot([xk, xk], [xk, yk], 'r-', 'LineWidth',1.2);
        % horizontal: (xk, g(xk)) -> (g(xk), g(xk))
        plot([xk, yk], [yk, yk], 'r-', 'LineWidth',1.2);
        % next iterate:
        xk = yk;
    end

    scatter(x0, 0, 40, 'filled', 'MarkerFaceColor',[0.1 0.6 0.1]);
    legend({'y=g(x)=-x^3','y=x','cobweb'}, 'Location','best');
end
```

Outputs:

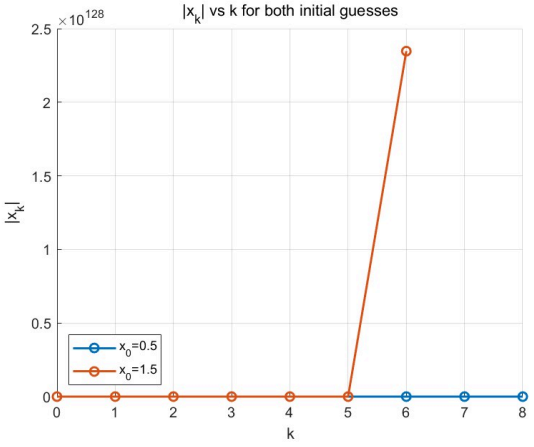
```
==== Case 1: x0 = 0.5 ====
    k        xk           abs_xk         f_xk
    _    __________    __________    __________

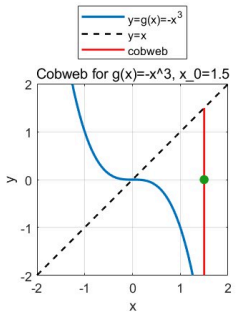
    0           0.5           0.5       0.44721
    1        -0.125         0.125      -0.12403
    2     0.0019531     0.0019531     0.0019531
    3    -7.4506e-09    7.4506e-09    -7.4506e-09
    4             0             0             0
```

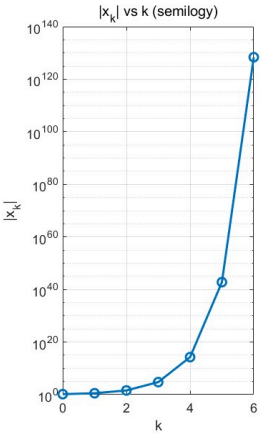
```
    5             0             0             0
    6             0             0             0
    7             0             0             0
    8             0             0             0

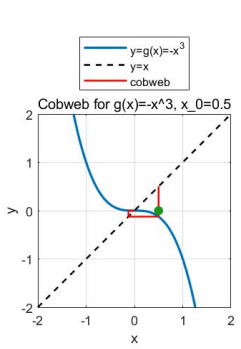

==== Case 2: x0 = 1.5 ====
    k        xk            abs_xk        f_xk
    _    __________    __________    _______

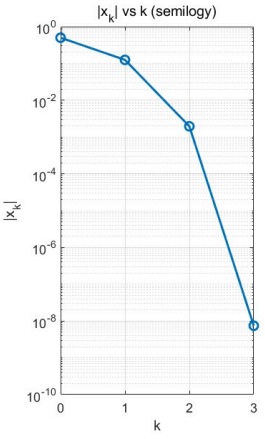
    0           1.5           1.5     0.83205
    1        -3.375         3.375     -0.9588
    2        38.443         38.443     0.99966
    3        -56815         56815          -1
    4     1.834e+14      1.834e+14           1
    5    -6.1684e+42     6.1684e+42          -1
    6    2.3471e+128    2.3471e+128           1
    7          -Inf            Inf         NaN
    8           NaN            NaN         NaN
```




---

# Problem 3: Stopping criterion for fixed point iteration

**Use an a-posteriori (successive-difference) test.**
Stop at the first index $K$ such that

$$|x_K - x_{K-1}| \le (1-q)\tau;$$

then the iterate already satisfies $|x^* - x_K| < \tau$.

**Why this works:**
Since $|\Phi'(x)| \le q < 1$ on $\mathbb{R}$, $\Phi$ is a contraction with constant $q$. For the fixed-point iteration $x_{k+1} = \Phi(x_k)$, the standard a-posteriori error bounds are

$$|x^* - x_k| \le \frac{q}{1-q}|x_k - x_{k-1}|$$

and

$$|x^* - x_k| \le \frac{1}{1-q}|x_{k+1} - x_k|.$$

These follow from contractivity and a geometric-series estimate of the tail $\sum_{j\ge k}|x_{j+1}-x_j|$. Therefore, if $|x_K-x_{K-1}|\le(1-q)\tau$, then

$$|x^*-x_K|\le\frac{q}{1-q}|x_K-x_{K-1}|\le q\tau<\tau,$$

and (even more directly) $|x^*-x_{K-1}|\le\frac{1}{1-q}|x_K-x_{K-1}|\le\tau$. These inequalities are standard consequences of the Banach contraction mapping theorem.

```
Dataview (inline field '=============== Results (tolerance = 1.0e-04) ================
True sqrt(3)  : 1.732050807568877

Steffensen    : p = 1.732050807568877,  |p - sqrt(3)| = 0.000e+00,  iterations = 3
Fixed-Point   : p = 1.732050810014727,  |p - sqrt(3)| = 2.446e-09,  iterations = 3
Bisection     : p = 1.731994628906250,  |p - sqrt(3)| = 5.618e-05,  iterations = 14
============================================================'): Error:
-- PARSING FAILED ------------------------------------------------

> 1 | =============== Results (tolerance = 1.0e-04) ================
    | ^
  2 | True sqrt(3)  : 1.732050807568877
  3 |

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string, variable
```

```
Dataview (inline field '=== Case 1: x0 = 0.5 ====
    k         xk            abs_xk          f_xk
    _    ___________    ___________    ___________

    0            0.5            0.5         0.44721
    1         -0.125          0.125        -0.12403
    2      0.0019531      0.0019531       0.0019531
    3    -7.4506e-09    7.4506e-09    -7.4506e-09
    4              0              0              0
    5              0              0              0
    6              0              0              0
    7              0              0              0
    8              0              0              0


==== Case 2: x0 = 1.5 ====
    k         xk            abs_xk         f_xk
    _    ___________    ___________    _______

    0            1.5            1.5     0.83205
    1         -3.375          3.375     -0.9588
    2         38.443         38.443     0.99966
    3         -56815          56815          -1
    4      1.834e+14      1.834e+14           1
    5    -6.1684e+42    6.1684e+42          -1
    6    2.3471e+128    2.3471e+128           1
    7           -Inf            Inf         NaN
    8            NaN            NaN         NaN'): Error:
-- PARSING FAILED ------------------------------------------------

> 1 | === Case 1: x0 = 0.5 ====
    | ^
  2 |     k         xk            abs_xk          f_xk
  3 |     _    ___________    ___________    ___________

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string, variable
```